

Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Miguel Veintimilla
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Grupal
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Luis Darío Sinche C
Fecha	Viernes, 2 de Octubre del 2026
Horario	11h30 – 13h30
Lugar	Laboratorio
Tiempo planificado en el Sílabo	2 horas

2. Objetivo(s) de la Práctica

- Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio. Objetivos específicos: Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD). Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos). Comparar el acceso mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución. Identificar en el código ensamblador generado las instrucciones lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).



3. Materiales, Reactivos

- No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi: 1 Arduino UNO (ATmega328P a 16 MHz). 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω . 1 pulsador.

4. Equipos y Herramientas

- Computador con navegador web y conexión a internet.
- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos. FEIRNNR - Carrera de Computación Microchip Studio 7 (solo Windows), con el simulador del ATmega328P.
- Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology). Cuenta de GitHub y acceso al EVA para la entrega.

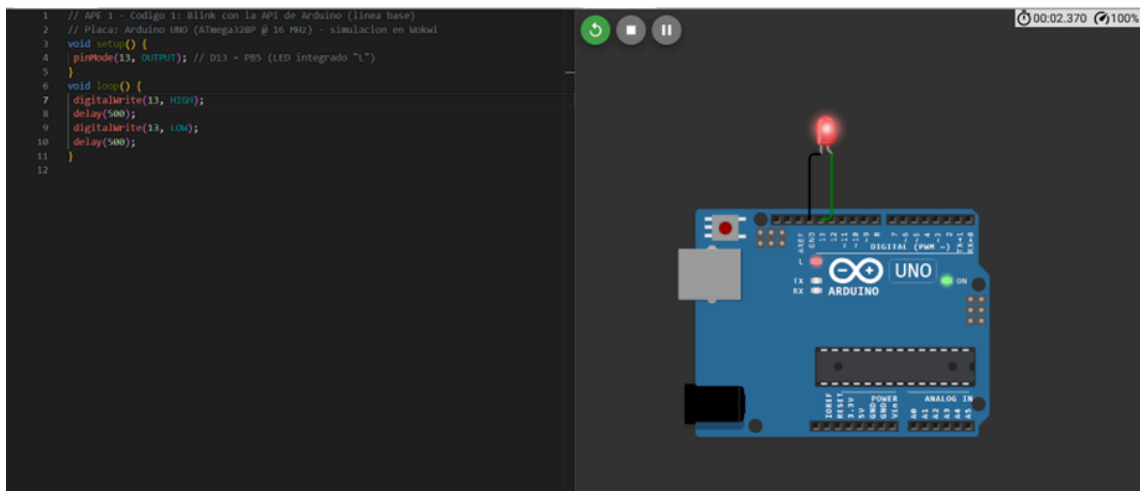
5. Procedimiento / Metodología

La práctica se desarrolló de manera progresiva a través de simulaciones virtuales y análisis de código, dividiéndose en las siguientes etapas metodológicas:

1. **Diseño y configuración del circuito virtual:** Utilizando la plataforma Wokwi, se implementó el esquema físico virtual conectando un Arduino UNO (microcontrolador ATmega328P) a 4 LEDs (rojo, amarillo, verde y azul) con sus respectivas resistencias limitadoras de corriente (220 Ω) y un pulsador físico.
2. **Control de salidas digitales mediante registros:** Se prescindió de las funciones nativas de Arduino (como pinMode) para acceder directamente a la arquitectura del microcontrolador. Se manipularon los registros de dirección de datos (DDRB, DDRD) para configurar los pines como salidas. Se programaron distintas secuencias de encendido en los LEDs escribiendo directamente en los registros de estado (PORTB, PORTD) mediante el uso de máscaras y operaciones de desplazamiento de bits (<<, >>).
3. **Operaciones lógicas y lectura de entradas:** Se desarrollaron cálculos manuales de operaciones a nivel de bit (AND, OR, XOR, NOT) y se corroboró su exactitud mediante su programación e impresión en el Monitor Serie. Posteriormente, se configuró el pin del pulsador como entrada habilitando la resistencia pull-up interna. Se implementó la lógica para leer su estado a través del registro PIND, permitiendo cambiar el modo (patrón de luces) de los LEDs en tiempo de ejecución.
4. **Evaluación comparativa de rendimiento temporal:** Se codificó una rutina de medición en el programa para calcular el tiempo exacto en microsegundos que tarda en ejecutarse el cambio de estado de un pin. Se contrastó el tiempo de ejecución de la API de alto nivel de Arduino frente al tiempo que toma la escritura directa en los registros del microcontrolador.
5. **Análisis de código a nivel ensamblador:** Empleando herramientas de compilación Microchip Studio, se tradujo el código C a código ensamblador. Se analizó el set de instrucciones resultante para identificar la correspondencia uno a uno entre las sentencias en C y las instrucciones del microcontrolador (tales como in, out, sbi, cbi, sbis, ori, eor), documentando también los ciclos de reloj que consume cada instrucción.

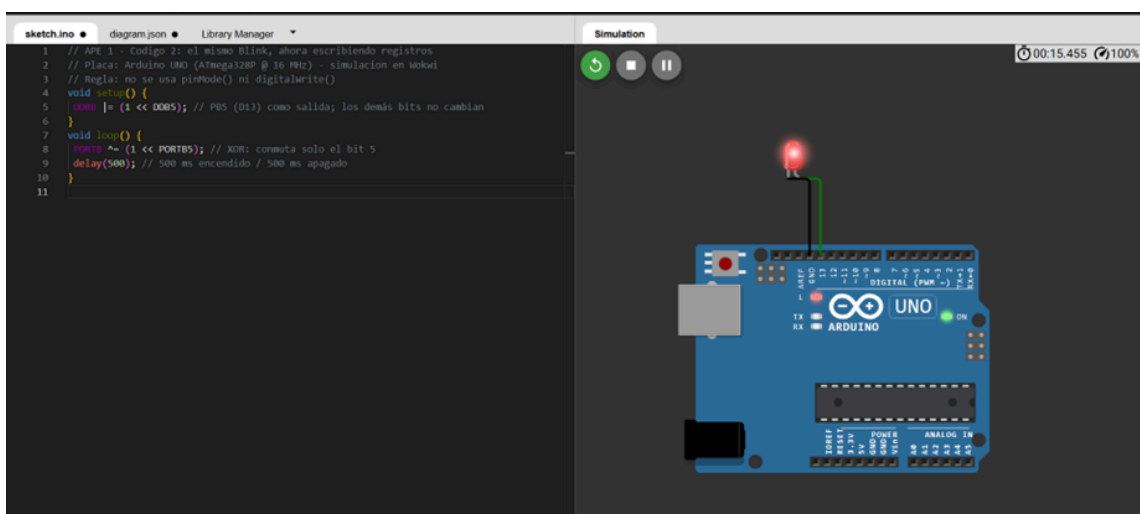
6. Resultados

A3:



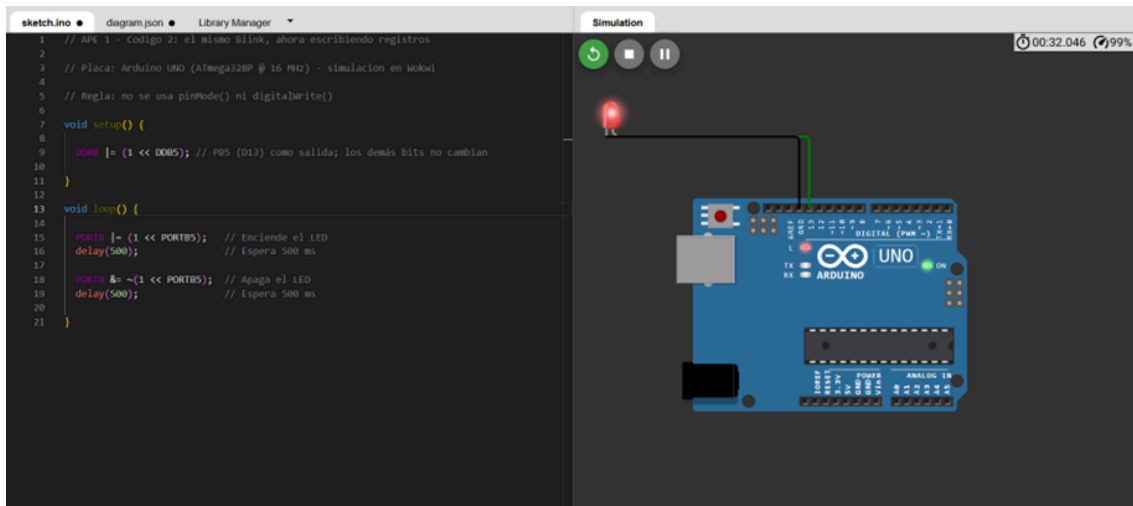
<https://wokwi.com/projects/476991897953560577>

B1:



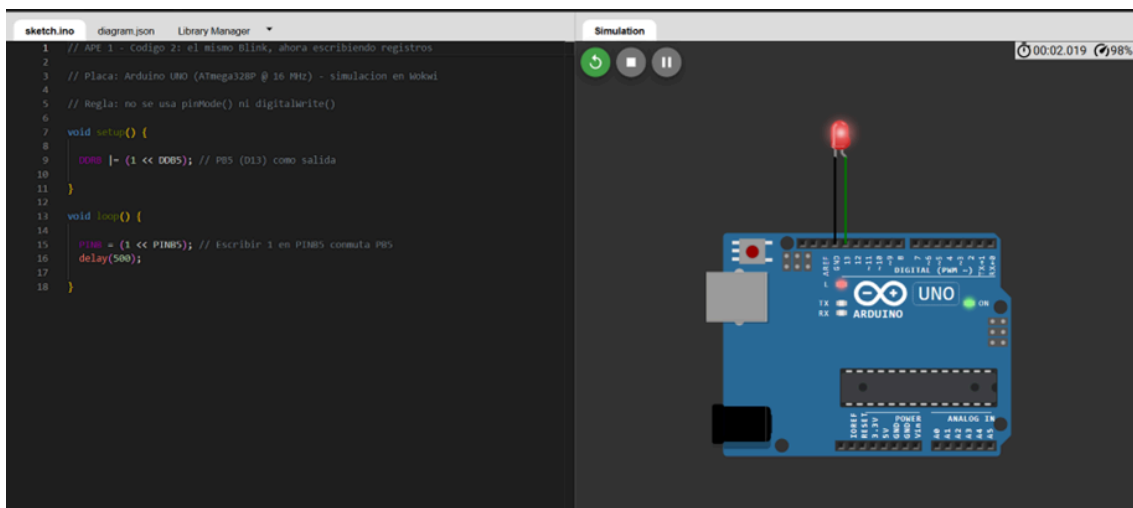
<https://wokwi.com/projects/476992043524228097>

B2:



<https://wokwi.com/projects/476992183082368001>

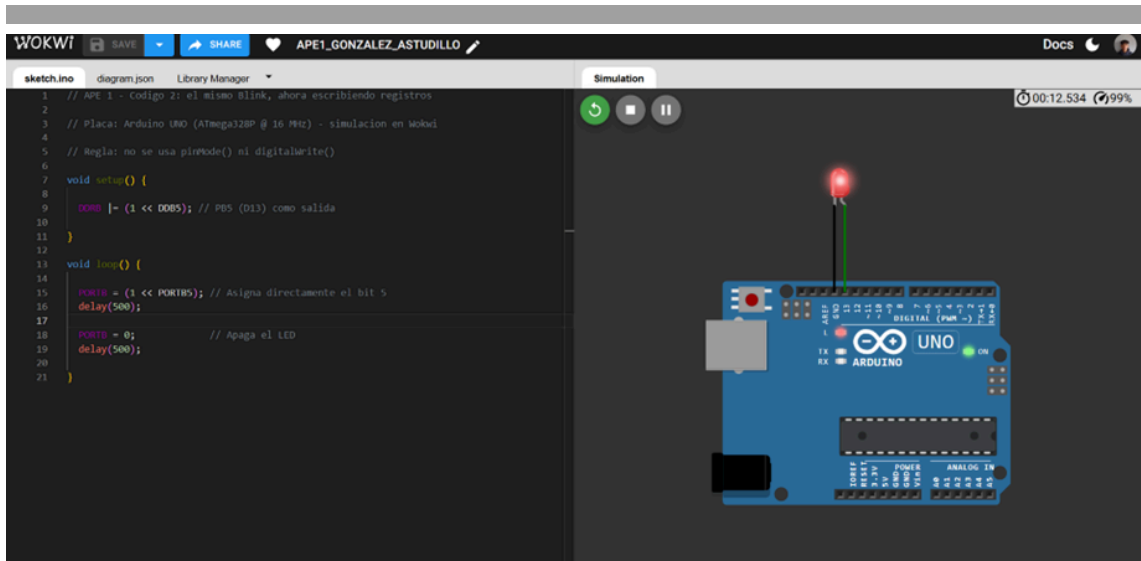
B3:



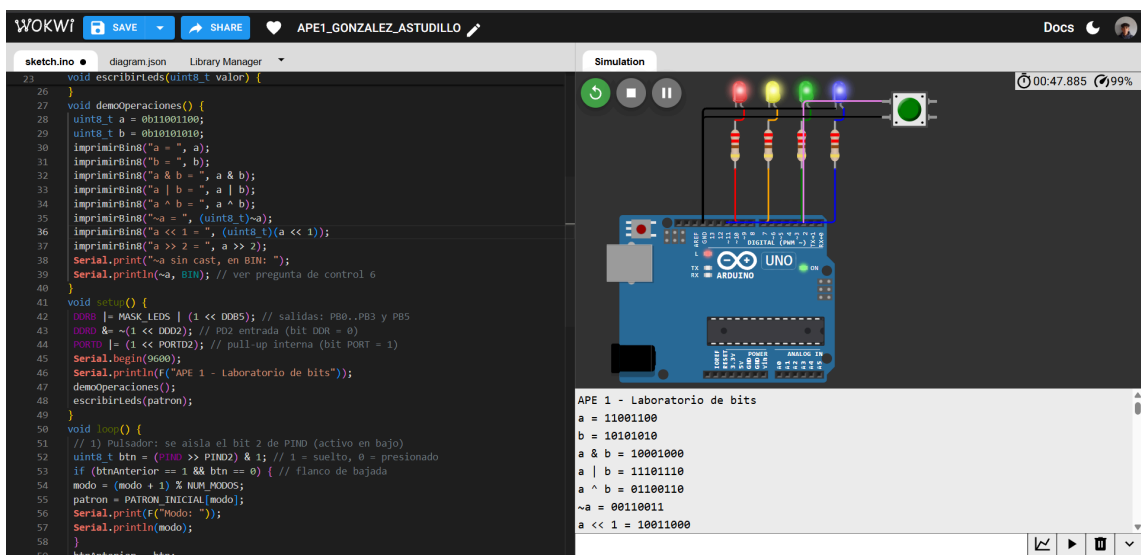
<https://wokwi.com/projects/476992357345234945>

La característica del ATmega328p que lo permite es que escribir un 1 en un bit de PINx conmuta el bit correspondiente de PORTx.

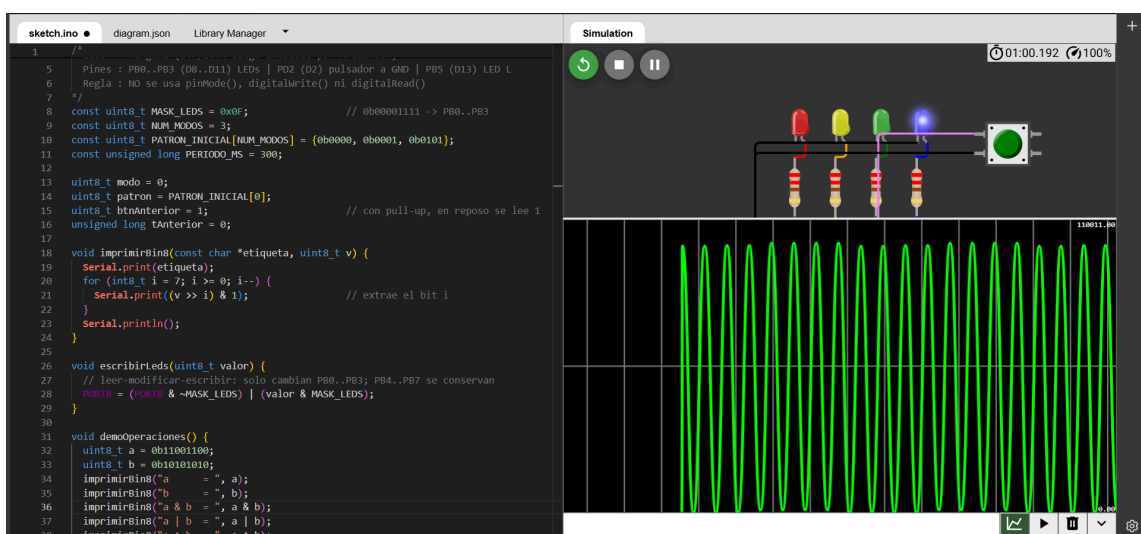
B4:



C2:



<https://wokwi.com/projects/476992581817067521>



<https://wokwi.com/projects/476992581817067521>

C3

Tenemos

$a = 11001100$
 $b = 10101010$

① AND $a \& b$

$$\begin{array}{r} 11001100 \\ \& 10101010 \\ \hline 10001000 \end{array}$$

② OR $a | b$

$$\begin{array}{r} 11001100 \\ | 10101010 \\ \hline 11101110 \end{array}$$

③ XOR $a \wedge b$

$$\begin{array}{r} 11001100 \\ \wedge 10101010 \\ \hline 01100110 \end{array}$$

④ Not $\sim a$

$a = 11001100$
 $\sim a = 00110011 //$

⑤ Desplazamiento $a \ll 1$

$11001100 \ll 1$

$$\begin{array}{r} 11001100 \\ 10011000 \end{array}$$

⑥ Desplazamiento $a \gg 2$

$11001100 \gg 2$

$11001100 \gg 1 = 01100110$
 $01100110 \gg 1 = \boxed{00110011} //$

C4:

MOD0 1:



WOKWI SAVE SHARE ♥ APE1_GONZALEZ_ASTUDILLO Docs 🌙 👤

sketch.ino diagram.json Library Manager

```
23 void escribirleds(uint8_t valor) {
24 }
25
26 }
27 void demoOperaciones() {
28   uint8_t a = 0b11001100;
29   uint8_t b = 0b10101010;
30   imprimirBin("a = ", a);
31   imprimirBin("b = ", b);
32   imprimirBin("a & b = ", a & b);
33   imprimirBin("a | b = ", a | b);
34   imprimirBin("a ^ b = ", a ^ b);
35   imprimirBin("~a = ", (~a));
36   imprimirBin("a << 1 = ", (uint8_t)(a << 1));
37   imprimirBin("a >> 2 = ", a >> 2);
38   Serial.print("a sin cast, en BIN: ");
39   Serial.println(~a, BIN); // ver pregunta de control 6
40 }
41 void setup() {
42   DDRC |= MASK_LEDS | (1 << DD0B5); // salidas: PB0..PB3 y PB5
43   DDRC &= ~(1 << DD0D2); // PD2 entrada (bit DD0 = 0)
44   PORTD |= (1 << PORTD2); // pull-up interna (bit PORT = 1)
45   Serial.begin(9600);
46   Serial.println(F("APE 1 - Laboratorio de bits"));
47   demoOperaciones();
48   escribirleds(patron);
49 }
50 void loop() {
51   // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
52   uint8_t btn = (PIND >> PIND2) & 1; // 1 = suelto, 0 = presionado
53   if (btnAnterior == 1 && btn == 0) { // flanco de bajada
54     modo = (modo + 1) % NUM_MODOS;
55     patron = PATRON_INICIAL[modo];
56     Serial.print(F("Modo: "));
57     Serial.println(modo);
58   }
59 }
```

Simulation

00:30.176 100%

a >> 2 = 0b110011
~a sin cast, en BIN: 1111111111111111111111111111111100110011
PORTB = 00100001
PORTB = 00000010
PORTB = 00100011
PORTB = 00000100
Modo: 1
PORTB = 00100010
PORTB = 00000100
PORTB = 00101000
PORTB = 00000001
PORTB = 00000010
PORTB = 00000100
PORTB = 00101000
PORTB = 00000001
PORTB = 00100010
PORTB = 00000100
PORTB = 00101000
PORTB = 00000001
PORTB = 00100010
PORTB = 00000100

MODO 2:

WOKWI SAVE SHARE ♥ APE1_GONZALEZ_ASTUDILLO Docs 🌙 👤

sketch.ino diagram.json Library Manager

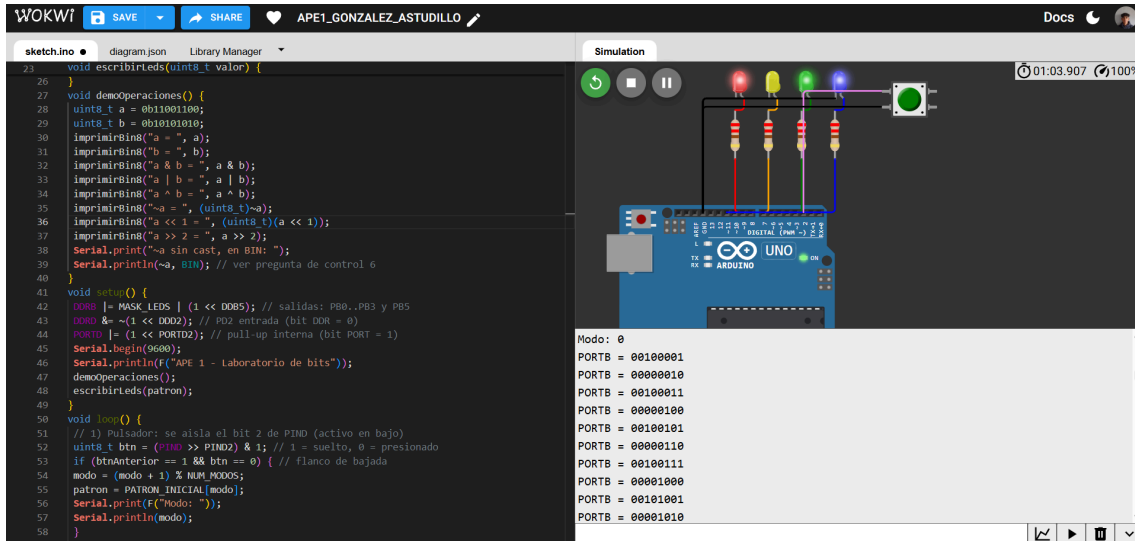
```
23 void escribirleds(uint8_t valor) {
24 }
25
26 }
27 void demoOperaciones() {
28   uint8_t a = 0b11001100;
29   uint8_t b = 0b10101010;
30   imprimirBin("a = ", a);
31   imprimirBin("b = ", b);
32   imprimirBin("a & b = ", a & b);
33   imprimirBin("a | b = ", a | b);
34   imprimirBin("a ^ b = ", a ^ b);
35   imprimirBin("~a = ", (~a));
36   imprimirBin("a << 1 = ", (uint8_t)(a << 1));
37   imprimirBin("a >> 2 = ", a >> 2);
38   Serial.print("a sin cast, en BIN: ");
39   Serial.println(~a, BIN); // ver pregunta de control 6
40 }
41 void setup() {
42   DDRC |= MASK_LEDS | (1 << DD0B5); // salidas: PB0..PB3 y PB5
43   DDRC &= ~(1 << DD0D2); // PD2 entrada (bit DD0 = 0)
44   PORTD |= (1 << PORTD2); // pull-up interna (bit PORT = 1)
45   Serial.begin(9600);
46   Serial.println(F("APE 1 - Laboratorio de bits"));
47   demoOperaciones();
48   escribirleds(patron);
49 }
50 void loop() {
51   // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
52   uint8_t btn = (PIND >> PIND2) & 1; // 1 = suelto, 0 = presionado
53   if (btnAnterior == 1 && btn == 0) { // flanco de bajada
54     modo = (modo + 1) % NUM_MODOS;
55     patron = PATRON_INICIAL[modo];
56     Serial.print(F("Modo: "));
57     Serial.println(modo);
58   }
59   btnAnterior = btn;
60 }
```

Simulation

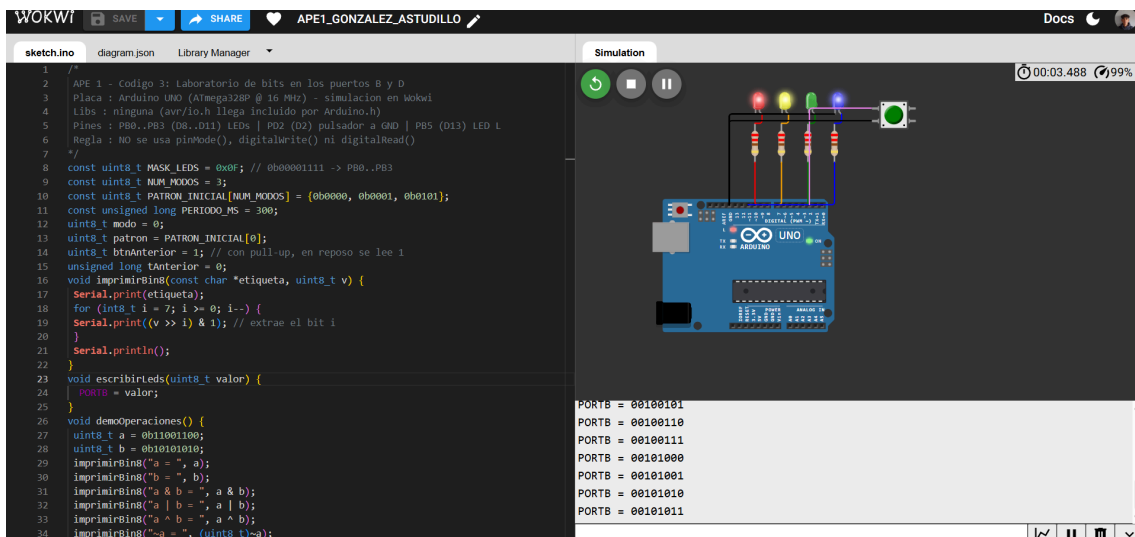
00:55.445 100%

PORTB = 00100010
Modo: 2
PORTB = 00001010
PORTB = 00100101
PORTB = 00001010
PORTB = 00100101
PORTB = 00001010
PORTB = 00100101
PORTB = 00001010
PORTB = 00100101
PORTB = 00001010

MODO 3:



C6:



Al sustituir la función `escribirLeds()` por `PORTB = valor;`, el LED “L” de D13 (PB5) deja de parpadear de manera adecuada. Esto sucede porque la asignación directa altera todos los bits del registro PORTB, incluyendo PB5, aunque el valor solo tiene el patrón dirigido a PB0–PB3. Por otro lado, la versión original emplea una operación de lectura-modificación-escritura que posibilita modificar solo PB0–PB3 y mantener el estado de los otros bits. Esto está vinculado con B4, donde se notó que `PORTB = (1 << PORTB5)` puede alterar las demás salidas del puerto.

C7:

PIN	FUNCION	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED 0 encendido	1	1	—

D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED “L” encendido	1	1	—

D1:

1ra ejecucion:

sketch.ino

diagram.json

Library Manager

```

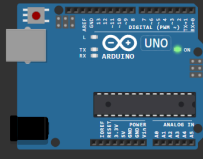
1 {
2   "version": 1, "author": "Luis Sinche", "editor": "wokwi",
3   "parts": [
4     {
5       "type": "wokwi-arduino-uno", "id": "uno", "top": 0, "left": 0, "attrs": {}
6     }
7   ]
8 }

```

Simulation

00:03.320

99%



```

digitalWrite: 3.395 us por escritura
Registro : 0.220 us por escritura

```

2da ejecucion

sketch.ino

diagram.json

Library Manager

```

1 // APE 1 - codigo 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
3
4 const uint16_t N = 10000;
5
6 void setup() {
7   Serial.begin(9600);
8   pinMode(13, OUTPUT);
9 }
10
11 unsigned long t0 = micros();
12 for (uint16_t i = 0; i < N; i++) {
13   digitalWrite(13, HIGH);
14   digitalWrite(13, LOW);
15 }
16 unsigned long t1 = micros();
17
18 for (uint16_t i = 0; i < N; i++) {
19   PORTB |= (1 << PORTB5);
20   PORTB &= ~(1 << PORTB5);
21 }
22 unsigned long t2 = micros();
23
24 Serial.print(F("digitalWrite: "));
25 Serial.print((t1 - t0) / (2.0 * N), 3);
26 Serial.println(F(" us por escritura"));
27
28 Serial.print(F("Registro : "));
29 Serial.print((t2 - t1) / (2.0 * N), 3);
30 Serial.println(F(" us por escritura"));
31
32 void loop() {}

```

Simulation

00:17.537

100%

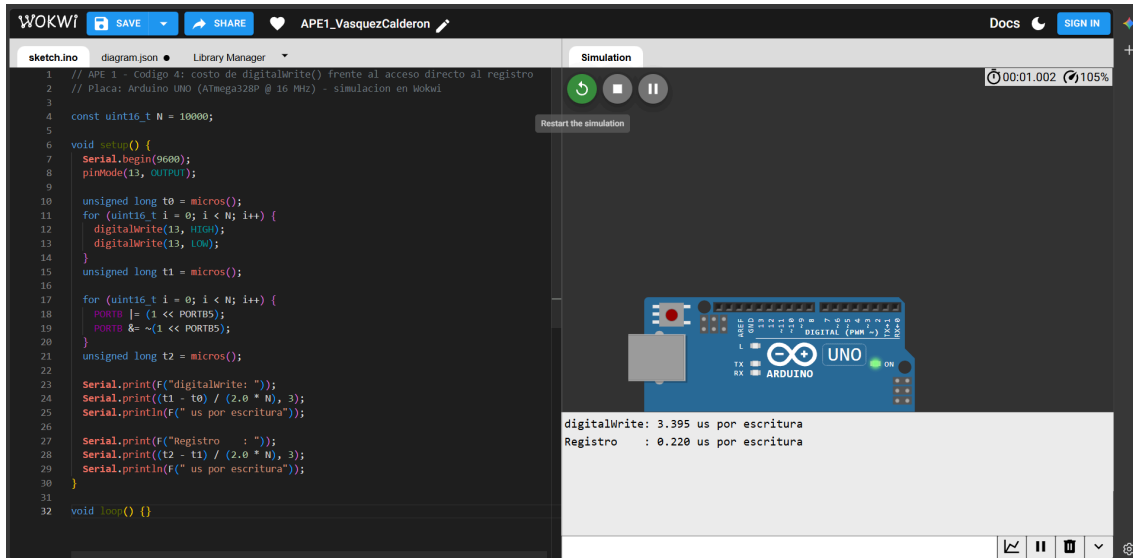


```

digitalWrite: 3.395 us por escritura
Registro : 0.220 us por escritura

```

3ra ejecucion



<https://wokwi.com/projects/476992668633920513>

Promedio final **3.395**

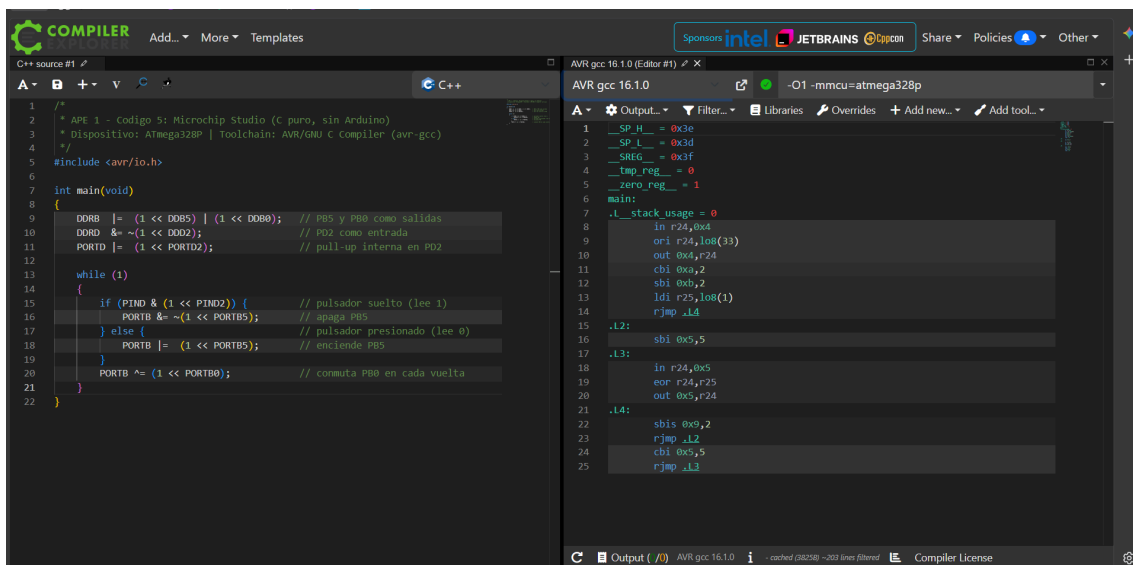


Tabla 4. Configuración de registros (para completar)

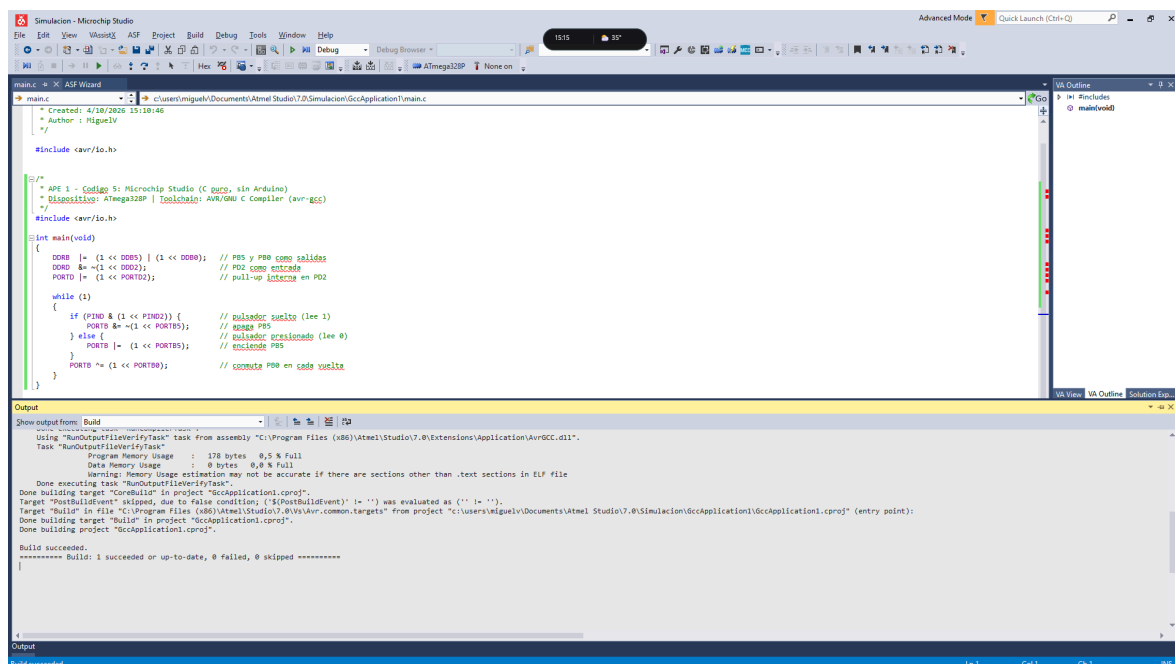
Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	0	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED "L" encendido	1	1	—

D2:

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in r24,0x4 ori r24,lo8(33) out 0x4,r24	3 (1 por instrucción)
DDRD &= ~(1 << DDD2);	cbi 0xa,2	2
PORTD = (1 << PORTD2);	sbi 0xb,2	2
if (PIND & (1 << PIND2))	sbis 0x9,2	1 o 2
PORTB &= ~(1 << PORTB5);	cbi 0x5,5	2
PORTB = (1 << PORTB5);	sbi 0x5,5	2
PORTB ^= (1 << PORTB0);	in r24,0x eor r24,r25 out 0x5,r24	3 (1 por instrucción)

E2:



```

main.c: 4/18/2026 15:18:46
Author: Higuely
#include <avr/io.h>

/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * @license: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
    DDRD &= ~(1 << DDD2); // PD2 como entrada
    PORTD |= (1 << PORTD2); // pull-up interno en PD2

    while (1)
    {
        if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5); // apaga PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
        PORTD ^= (1 << PORTD0); // conmuta PD0 en cada vuelta
    }
}
  
```

Build succeeded.

El código se ejecutó y compiló sin ningún error

E3:

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in, ori, out	3
DDRD &= ~(1 << DDD2);	in, andi, out	3
PORTD = (1 << PORTD2);	in, ori, out	3
if (PIND & (1 << PIND2))	sbis, rjmp	2



PORTB &= ~ (1 << PORTB5);	in, andi, out, rjmp	4
PORTB = (1 << PORTB5);	in, ori, out	3
PORTB ^= (1 << PORTB0);	in, ldi, eor, out	4

The screenshot shows the AVR Studio IDE with the following components:

- Top Menu Bar:** File, Edit, View, Windows, Project, Build, Debug, Tools, Window, Help.
- Top Toolbar:** Standard IDE tools for file operations, editing, and debugging.
- Project Explorer (Left):** Shows the project structure with files like main.c and sub1.c.
- Main Editor (Center):** Displays the code for main.c, which includes comments in Spanish and C code for an AVR microcontroller. The code sets up pins PB0, PB1, and PB5, and implements a loop that toggles PB0 and PB1 based on a button press (PB5).
- I/O Window (Right):** A window titled "I/O" showing a list of hardware peripherals. The "IO Port (PORTB)" is selected, and a table below it shows the bit configuration for PORTB, DDRB, and PORTC.

Code in main.c:

```

/*
 * gccApplications.c
 *
 * Created: 4/18/2026 15:18:46
 * Author : Higuely
 */

#include <avr/io.h>

/*
 * APE 1 - Codigo 5; Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRC |= (1 << DDRC5); // PB0 y PB1 como salidas
    DDRC &= ~(1 << DDRC2); // PB2 como entrada
    PORTD |= (1 << PORTD2); // pull-up interna en PB2

    while (1)
    {
        if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
            PORTB |= (1 << PORTB5); // apaga PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
        PORTB ^= (1 << PORTB0); // conmute PB0 en cada vuelta
    }
}

```

I/O Window Details:

- Peripherals List:** Analog Comparator (AC), Analog-to-Digital Converter (ADC), CPU Registers (CPU), EEPROM (EEPROM), External Interrupts (EXT), IO Port (PORTB), IO Port (PORTC), IO Port (PORTD), Serial Peripheral Interface (SPI), Timer/Counter, 16-bit (TC).
- Selected Peripheral: IO Port (PORTB)**
- Table:**

Name	Address	Value	Bits
PORTB	0x21	0x00	0 0 0 0 0 0 0 0
DDRB	0x24	0x21	0 0 0 0 0 0 0 0
PORTC	0x25	0x20	0 0 0 0 0 0 0 0



1859



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

The image displays two screenshots of the Microchip Studio IDE, showing the simulation of an AVR microcontroller. The top screenshot shows the code in 'main.while.c' and the I/O window with the 'PORTB' register selected. The bottom screenshot shows the same code but with the 'PORTB' register selected in the I/O window, and the 'PORTB' register value is updated to 0x21.

Top Screenshot:

```
main.while.c
/*
 * BcApplication1.c
 * Created: 4/18/2026 15:18:46
 * Author : MiguelV
 */

#include <avr/io.h>

/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRC |= (1 << DDRC5) | (1 << DDRC6); // PB5 y PB6 como salidas
    DDRC &= ~(1 << DDRC2); // PB2 como entrada
    PORTD |= (1 << PORTD2); // pull-up interna en PD2

    while (1)
    {
        if (PINB & (1 << PINB2)) { // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5); // apaga PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
        PORTB ^= (1 << PORTB6); // conmuta PB6 en cada vuelta
    }
}
```

I/O Window (Top):

Name	Address	Value	Bits
PINB	0x23	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x25	0x20	00000000

Bottom Screenshot:

```
main.while.c
/*
 * BcApplication1.c
 * Created: 4/18/2026 15:18:46
 * Author : MiguelV
 */

#include <avr/io.h>

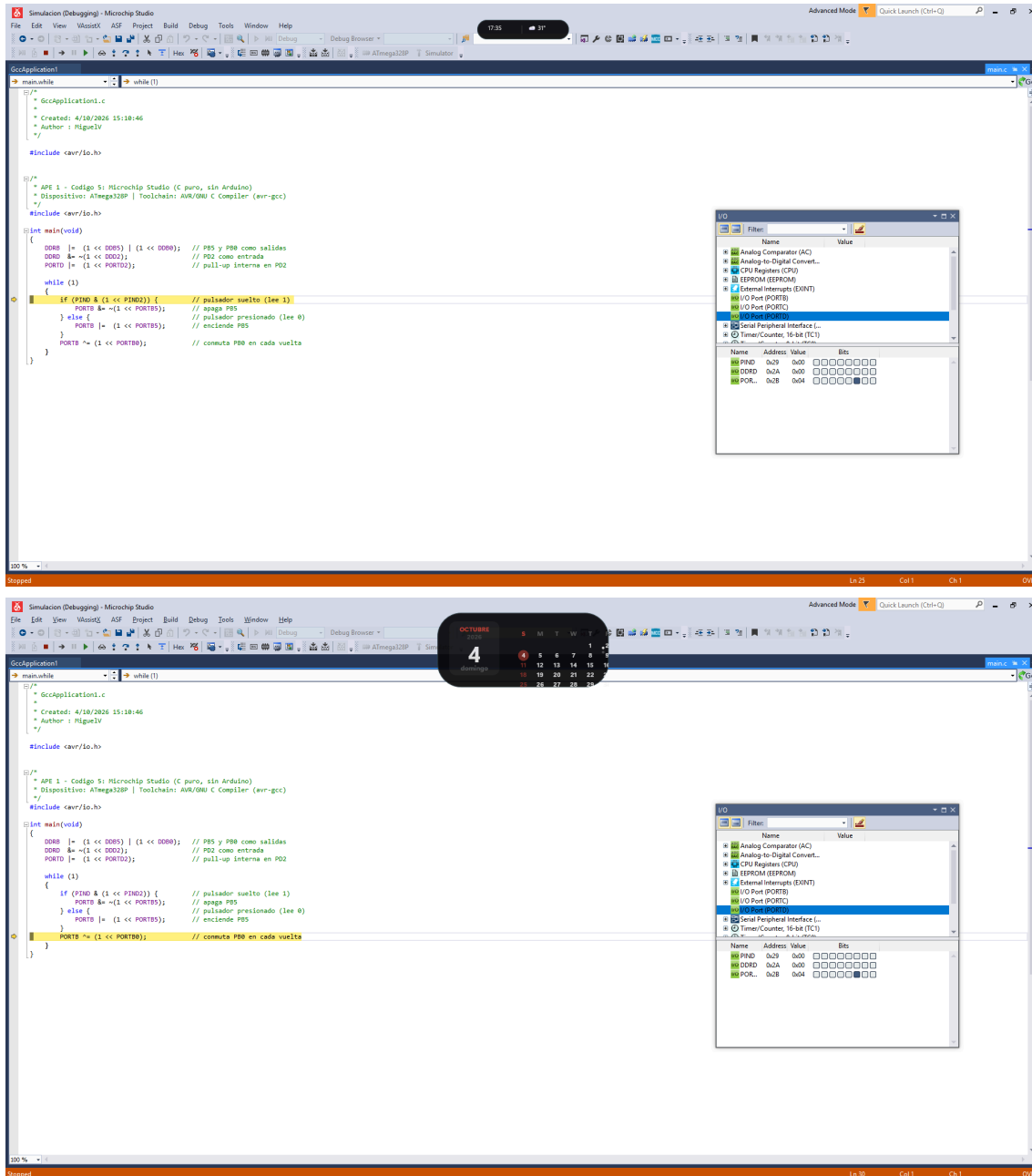
/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRC |= (1 << DDRC5) | (1 << DDRC6); // PB5 y PB6 como salidas
    DDRC &= ~(1 << DDRC2); // PB2 como entrada
    PORTD |= (1 << PORTD2); // pull-up interna en PD2

    while (1)
    {
        if (PINB & (1 << PINB2)) { // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5); // apaga PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
        PORTB ^= (1 << PORTB6); // conmuta PB6 en cada vuelta
    }
}
```

I/O Window (Bottom):

Name	Address	Value	Bits
PINB	0x23	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x25	0x21	00000001



E6:

7. Preguntas de Control

1. ¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los bloques del ATmega328P que identificó en la práctica.

Un microprocesador como el de mi compu contiene solo la unidad de procesamiento (CPU) y necesita que le conectes memoria y periféricos externos para funcionar. Un microcontrolador como el ATmega328P es una "computadora en un solo chip": ya integra la CPU, la memoria (RAM y Flash) y los puertos de entrada/salida (I/O) en la misma pastilla.



2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro F(...) en el Código

Implica que el microcontrolador tiene memorias y buses de comunicación físicamente separados para los Datos (RAM) y para las Instrucciones/Programa (Flash). Esto le permite a la CPU leer una instrucción y un dato al mismo tiempo, haciéndolo más rápido y por lo que la memoria RAM del ATmega328P es pequeñísima (apenas 2 KB). Por defecto, al imprimir texto normal, el sistema copia ese texto de la Flash a la RAM, desperdiciando espacio. La macro `F()` aprovecha la separación de la arquitectura Harvard para decirle al sistema: "deja este texto guardado en la memoria Flash y léelo desde ahí", ahorrando RAM valiosa.

3. Explique por qué $\text{PORTB} = (1 \ll \text{PORTB5})$; puede alterar otras salidas y $\text{PORTB} |= (1 \ll \text{PORTB5})$; no, usando lo observado en los pasos B4 y C6.

La asignación directa (`PORTB = valor`) altera todos los bits del registro PORTB al mismo tiempo, modificando pines que no querías cambiar. Por otro lado, usar el operador `|=` realiza una operación de lectura-modificación-escritura que permite cambiar solo el bit deseado (como PB5) y mantener el estado actual de los demás bits intacto.

4. Para usar PD2 como entrada con pull-up, ¿qué valores deben tener DDRD2 y PORTD2? ¿Qué se lee en PIND2 con el pulsador suelto y presionado, y por qué?

Según tu Tabla 4 y la sección C7, el bit DDRx debe ser 0 y el bit PORTx debe ser 1. Cuando el pulsador está suelto, el bit PINx leído es 1.

Cuando el pulsador está presionado, el bit PINx leído es 0.

5. Calcule a mano $0b11001100$ & $0b10101010$, $|$ y $\wedge$, y verifique sus resultados con el Monitor Serie.

Me dieron los mismos resultados al calcular a mano y en el monitor serie.

6. Si `a` es de tipo `uint8_t`, ¿por qué `Serial.println(~a, BIN)` imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?

Como se observa en la consola de tu simulación (Modo 1), imprimir ~a sin "cast" provoca que se muestren más bits (se imprime 11111111111111111111111100110011). El Código 3 lo corrige aplicando un cast de esta forma: (uint8_t)~a.

7. Con los datos de la Parte D, explique por qué `digitalWrite()` es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?

Según las capturas de la Parte D1, `digitalWrite()` tomó en promedio 3.395 us por escritura, mientras que el acceso directo a los registros tomó solo 0.220 us por escritura. El acceso a registros es

muchísimo más rápido porque escribe directamente en el hardware.

8. ¿Qué instrucción AVR generó $\text{PORTB} |= (1 \ll \text{PORTB5});$ y cuántos ciclos toma?

De acuerdo con la "Tabla 5" en el paso E3, generó las instrucciones `in`, `ori`, `out` y toma 3 ciclos. (Nota: En la Tabla 5 de la parte D2, también indicaste que se generó mediante `sbi 0x5, 5` tomando 2 ciclos).

¿Por qué $\text{PORTB} ^= (1 \ll \text{PORTB0});$ no se tradujo a una sola instrucción?

Según la Tabla 5 del paso E3, esta línea en C se tradujo a las instrucciones AVR `in`, `ldi`, `eor`, `out`, tomando un total de 4 ciclos. No se hizo en una sola instrucción porque requiere varios pasos a nivel ensamblador para aplicar la operación lógica XOR (cargar el valor, realizar la operación exclusiva y sacarlo).

9. Sostenibilidad: indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.

Simular en Wokwi antes de armar el circuito real evita que quemes componentes físicos (LEDs, resistencias o el propio microcontrolador) por cortocircuitos o malas conexiones, lo que significa menos placas arruinadas en la basura.

10. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).

Es totalmente seguro e ideal para probar la lógica. No hay cables falsos que hagan mal contacto, no necesitas comprar hardware para aprender y puedes compartir el circuito con un enlace.

8. Conclusiones

- Durante esta práctica comprobé que manipular directamente los registros (DDR, PORT, PIN) del ATmega328P es drásticamente más rápido que usar la API nativa de Arduino. Medir los tiempos de ejecución me demostró que el acceso a registros toma apenas 0.220 microsegundos por escritura, frente a los 3.395 microsegundos de `digitalWrite`, un rendimiento que será clave cuando necesite programar aplicaciones donde el tiempo de respuesta sea crítico.
- El uso del simulador y de Microchip Studio me permitió entender lo que realmente ocurre dentro del microcontrolador a nivel de hardware. Observar cómo mis líneas de código en C se traducen a instrucciones específicas de ensamblador (como `sbi`, `cbi`, `in`, `out`), y ver cómo cambian los bits en la memoria paso a paso, me dio una perspectiva mucho más profunda sobre cómo optimizar mis programas en lugar de solo escribir código a ciegas.



9. Bibliografía

[1] W. P. Chamba Zaragocín, E. R. Guamán Quinche, F. J. Alvarez Pineda, J. O. Guamán Quinche, G. B. Zaragocín Arrobo y G. Á. Valarezo Benítez, Análisis y diseño preliminar de software basado en el modelado de requisitos y casos de uso. Un enfoque teórico y práctico. Loja, Ecuador: Universidad Nacional de Loja, 2025. Disponible en:
<https://unl.edu.ec/investigacion/produccion-cientifica/analisis-y-diseno-preliminar-de-software-basado-en-el-modelado>